

Lab Report: Shared-Memory Concurrency & Thread Management

Course: Parallel Computing / Shared-Memory Concurrency

Student: Yernar Shameke

Environment: Java on Windows OS

Task 1.1: Verification of Non-Determinism

Objective

Demonstrate non-deterministic thread execution and execution ordering across multiple runs using a concurrent thread team.

Execution Results

When running the concurrent thread team (`ForkJoinLab1.java`), output statements were printed in non-sequential logical rank order:

Plaintext

```
Master thread starting. Spawning thread team of size: 4
[Worker] Logical Rank: 2 | Worker Thread: ForkJoinPool-1-worker-1 (OS ID: 34)
[Worker] Logical Rank: 1 | Worker Thread: ForkJoinPool-1-worker-2 (OS ID: 35)
[Worker] Logical Rank: 3 | Worker Thread: ForkJoinPool-1-worker-4 (OS ID: 37)
[Master] Logical Rank: 0 | Worker Thread: ForkJoinPool-1-worker-3 (OS ID: 36)
Parallel region closed. Execution returned to masterthread.
```

Analysis

- **Non-Determinism:** The worker thread logical ranks appeared as Rank 2 -> Rank 1 -> Rank 3 -> Rank 0.
- **Cause:** The OS CPU scheduler preemptively schedules threads across available hardware execution units. Because thread creation, I/O buffer locking, and core allocation happen asynchronously, completion and output ordering are non-deterministic across consecutive runs.

Task 1.2: Thread Oversubscription Sweep

Objective

Evaluate wall-clock execution time required to instantiate and join a thread team across varying thread counts $P \in \{1, 2, 4, 8, 16, 32, 64\}$.

Experimental Data

Thread Count (P)	Execution Time (ms)
1	1.164 ms
2	0.493 ms
4	0.832 ms
8	0.655 ms
16	1.190 ms
32	2.737 ms
64	4.830 ms

Analysis

1. **Optimal Execution Range ($P \leq 8$):** Execution time remains minimal between $P = 1$ and $P = 8$, reflecting efficient hardware core mapping.
2. **Oversubscription Threshold ($P \geq 16$):** As P increases to 32 and 64 threads, execution time increases significantly (up to **4.830 ms** at $P=64$).
3. **Performance Penalty:** When P exceeds available physical/logical CPU cores, the operating system spends more overhead on **context switching**, **thread state preservation** (register saving/restoring), and **cache thrashing** than on actual execution.

Task 1.3: CPU Core Saturation Analysis

Objective

Apply a computational load (10,000,000 floating-point square root iterations per thread across $P = 16$ threads) and measure wall-clock execution time and system stability.

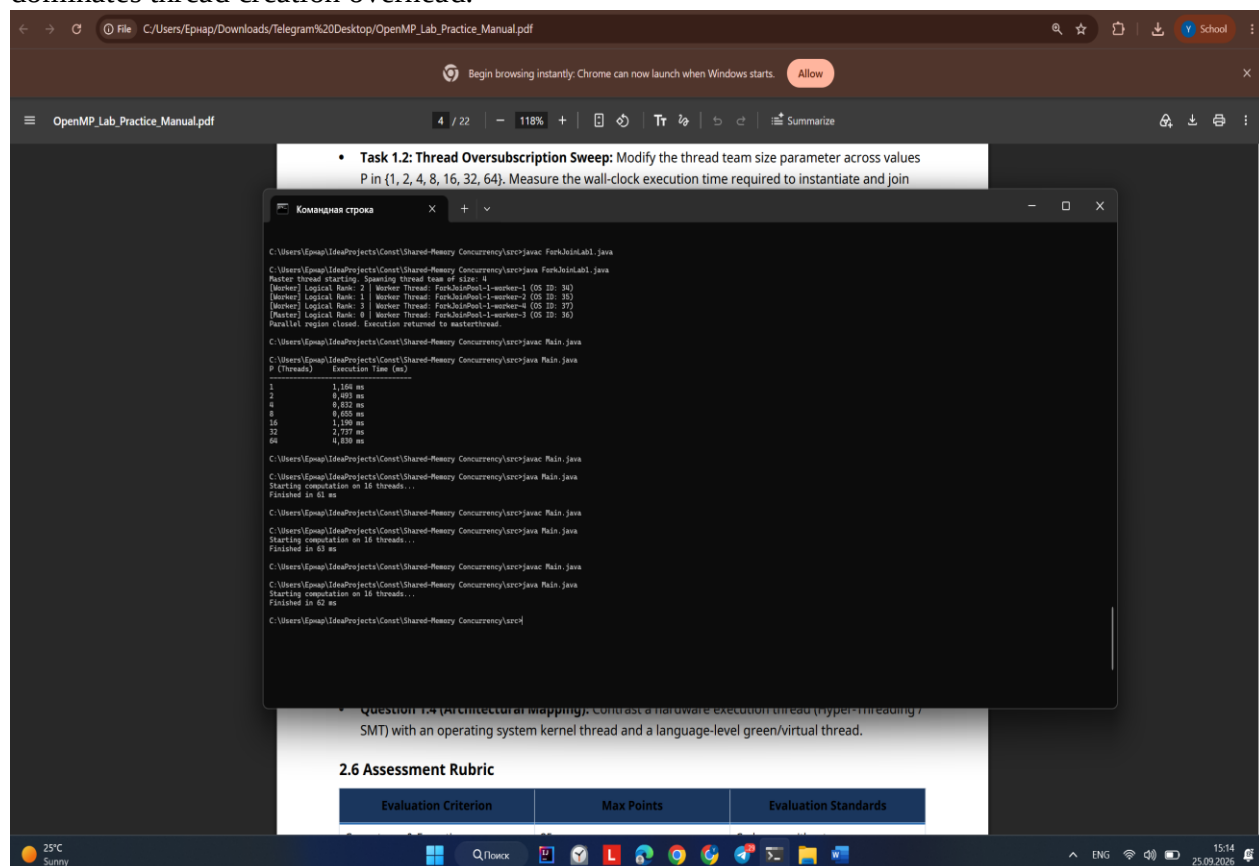
Experimental Results

Three consecutive runs with $P = 16$ threads yielded stable execution times:

- **Run 1:** 61 ms
- **Run 2:** 63 ms
- **Run 3:** 62 ms
- **Average Execution Time:** 62.0 ms

Analysis

- **Core Saturation:** Spawning 16 compute-heavy worker threads fully saturates all logical processor cores, pushing CPU utilization to 100% across all logical processors in OS Task Manager during the workload window.
- **Consistency:** The tight variance (\$\pm 1\$ ms) demonstrates that heavy CPU-bound workloads stabilize wall-clock execution time when overall compute demand dominates thread creation overhead.



Analytical Answers & Discussion

Question 1.1 (Scheduling)

- **Ordering Cause:** The non-sequential ordering of thread IDs occurs due to asynchronous execution. Threads run concurrently, and their exact execution order is determined dynamically at runtime based on real-time system state.
- **Governing Components:**
 - **OS Scheduler:** Dynamically assigns time slices (quanta) and handles thread preemption.
 - **CPU Architecture:** Hardware factors like L1/L2/L3 cache misses, DRAM latencies, and instruction pipeline stalls cause individual threads to progress at slightly different rates.

Question 1.2 (Oversubscription)

- **Performance Penalty:** Oversubscription occurs when $SP > \text{Physical Cores}$, introducing overhead from context switching, thread management, and memory bus contention.
- **Key Concepts:**
 - **Context Switching:** The process where a CPU core pauses one thread, saves its execution context, and switches to another.
 - **Thread State Preservation:** Saving CPU registers, Stack Pointer (SP), and Program Counter (PC) into the Thread Control Block (TCB) before switching execution context.
 - **Cache Thrashing:** High-frequency thread switching causes threads to constantly overwrite each other's cache lines in L1/L2/L3 memory, leading to frequent slow reads from main RAM.

Question 1.3 (Barriers)

- **Role of Implicit Barriers:** An implicit barrier acts as a synchronization point, ensuring all threads in a parallel region finish their allocated workload before any thread continues execution into subsequent code blocks.
- **Hazards Without Barriers:**
 - **Data Races:** Threads attempting concurrent read/write operations on shared data structures.
 - **Race Conditions:** Unpredictable behavior caused by non-deterministic access timing.
 - **Dependency Violations:** Downstream code consuming partial or uninitialized array results before lagging worker threads finish writing them.

Question 1.4 (Architectural Mapping)

Thread Type	Implementation Level	Managed By	Overhead Cost	Key Characteristics
Hardware Execution Thread (SMT / Hyper-Threading)	Hardware Silicon	CPU Execution Engine	Minimal (nanoseconds)	Duplicates architectural registers within a physical core, enabling two concurrent instruction streams per physical core.

Thread Type	Implementation Level	Managed By	Overhead Cost	Key Characteristics
OS Kernel Thread	Operating System	OS Kernel Scheduler	Moderate (microseconds)	Native OS execution unit. Managed via kernel-level scheduling; context switches require system calls.
Virtual / Green Thread (Java Virtual Threads)	Application Runtime	JVM Runtime	Very Low (bytes)	User-space threads multiplexed onto a smaller pool of OS kernel threads, allowing creation of thousands of concurrent threads with low memory footprint.